

Assignment 4

This exercise sheet is mainly devoted to the Cholesky and the QR decompositions. In the graded exercise, you will also learn how to compare runtimes, which is useful to know in general.

Graded exercise

Exercise 16 (Cholesky decomposition and comparison with LU)

This exercise is divided in two (independent) parts: (a)-(b) and (c).

- (a) Implement a function `cholesky_decomposition` that, given a symmetric positive definite matrix $A \in \mathbb{R}^{m \times m}$ in input, computes its Cholesky decomposition and returns the lower triangular matrix L_* such that $A = L_* L_*^\top$.
- (b) Consider the matrix

$$A = \begin{pmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{pmatrix},$$

and compute by hand its Cholesky decomposition. Compare then your result with the answer of your routine from (a) when called for this matrix A .

- (c) We now compare experimentally the running times of computing the LU - and the Cholesky decompositions. Since running times depend heavily on the implementation, to do the comparison we do not use our self-implemented routines but the built-in routines of the `scipy` module instead, which are optimized. Consider the tridiagonal matrix $A \in \mathbb{R}^{m \times m}$ given by

$$A = \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix},$$

which can be checked to be positive definite. Write a PYTHON script that, for $m = 2^i$ with $i = 10, 11, 12, 13, 14$, calls the `scipy` routines `linalg.cholesky` and `linalg.lu` to compute the Cholesky and LU -decompositions of A , respectively. For each m , measure the running time of computing each of the decompositions, and at the end produce a `loglog` plot (logarithmic scale on both the x - and y -axes) of running times versus m for the two decompositions. Motivate the result in the plot with what you know about the computational complexity of the two algorithms.

Hint: To set up the matrix A , you can use the `numpy` function `diag`.

Hint: To measure the running times, you can use the function `time` of the PYTHON module `time`.

Other exercises

Exercise 17 (Condition of the normal equations)

(a) For $0 < \varepsilon \ll 1$, we consider the matrix A and the vector b given by

$$A = \begin{pmatrix} 1 & 1 \\ \varepsilon & 0 \\ 0 & \varepsilon \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ \varepsilon \\ \varepsilon \end{pmatrix}.$$

- (i) Compute the solution to the least squares problem $\min_{w \in \mathbb{R}^2} \|b - Aw\|_2^2$ using the normal equations.
- (ii) Why is it not a good idea, for this problem, to solve the normal equations numerically?
- (iii) What happens for $\varepsilon = 0$?

(b) We consider the *full* QR -decomposition of a matrix $A \in \mathbb{R}^{m \times n}$, given by

$$A = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_1 \\ 0 \end{pmatrix},$$

where $A = Q_1 R_1$ is the *reduced* QR -decomposition. Let $v = \arg \min_{w \in \mathbb{R}^n} \|b - Aw\|_2$ be the solution to the least squares problem.

- (i) Show that $r = b - Av = (I - Q_1 Q_1^\top) b = Q_2 Q_2^\top b$.
- (ii) Which of the three expressions above should one use to compute numerically the residual r ? Why?

Exercise 18 (Computing the QR decomposition)

Consider the matrix

$$A = \begin{pmatrix} -2 & -1 + 3\sqrt{3} \\ 2\sqrt{3} & 3 + \sqrt{3} \end{pmatrix}.$$

Compute (by hand) the QR -decomposition of A using a Householder transformation (note that here we don't have to distinguish between full or reduced QR because the matrix is square). In the transformation, normalize the normal vector w such that its first component is 1, $w_1 = 1$.